

LAPORAN TEKNIS TUGAS PROYEK APLIKASI KRIPTOGRAFI

Topik A — Enkripsi dengan Algoritma Modern

BRANKAS BERKAS PRIBADI TERENKRIPSI BERBASIS WEB MENGUNAKAN AES-256-GCM DAN ChaCha20-Poly1305

Nama	NPM	Kontribusi Utama
« isi nama anggota 1 »	« NPM »	Modul krypto inti dan unit test
« isi nama anggota 2 »	« NPM »	Antarmuka web dan integrasi API
« isi nama anggota 3 »	« NPM »	Skrip pengujian, analisis, dan laporan

Keterangan	Tautan / Isi
Repositori GitHub	« https://github.com/... »
Video demonstrasi	« https://youtu.be/... »
Mata kuliah	Keamanan Informasi (Information Security)
Dosen pengampu	Ir. Alam Rahmatulloh, S.T., M.T., MCE., IPM.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS SILIWANGI
2026

1. Pendahuluan

1.1 Latar Belakang

Data pribadi seperti Nomor Induk Kependudukan, nomor rekening, dan salinan dokumen identitas kini tersimpan di banyak perangkat dan layanan penyimpanan awan. Ketika perangkat hilang atau penyimpanan awan disusupi, berkas yang tersimpan dalam bentuk polos dapat langsung terbaca oleh pihak yang tidak berhak. Enkripsi pada sisi pengguna menjadi lapisan pertahanan terakhir karena berkas tetap tidak terbaca walaupun media penyimpanannya jatuh ke tangan orang lain.

Persoalannya, banyak aplikasi enkripsi sederhana hanya menjamin kerahasiaan tanpa menjamin keutuhan. Mode operasi klasik seperti Electronic Codebook (ECB) bahkan membocorkan pola plainteks, sedangkan Cipher Block Chaining (CBC) tanpa Message Authentication Code membiarkan cipherteks diubah tanpa terdeteksi. Kesalahan lain yang sering muncul adalah memakai kata sandi pengguna langsung sebagai kunci, padahal entropi kata sandi manusia jauh di bawah 256 bit.

1.2 Rumusan Masalah

- Bagaimana merancang aplikasi enkripsi berkas yang menjamin kerahasiaan sekaligus keutuhan data, sehingga setiap perubahan pada cipherteks pasti terdeteksi?
- Bagaimana menurunkan kunci 256 bit yang kuat dari kata sandi pengguna yang entropinya rendah?
- Bagaimana membuktikan secara kuantitatif bahwa cipherteks yang dihasilkan tidak dapat dibedakan dari data acak, dan algoritma mana yang lebih sesuai pada perangkat tertentu?

1.3 Tujuan

- Membangun aplikasi web yang mampu mengenkripsi dan mendekripsi teks maupun berkas dengan AES-256-GCM dan ChaCha20-Poly1305.
- Menerapkan turunan kunci berbasis kata sandi memakai scrypt, PBKDF2, atau Argon2id dengan salt acak, serta nonce acak untuk setiap operasi enkripsi.
- Menguji aplikasi secara kuantitatif melalui uji kebenaran, waktu eksekusi, avalanche effect, entropi, dan uji chi-square, lalu menganalisis hasilnya.
- Membandingkan dua algoritma AEAD modern dan menjelaskan kapan masing-masing lebih tepat dipakai.

2. Dasar Teori

2.1 Authenticated Encryption with Associated Data (AEAD)

AEAD adalah skema enkripsi yang dalam satu operasi menghasilkan cipherteks sekaligus authentication tag. Secara formal, fungsi enkripsi dan dekripsi AEAD dinyatakan sebagai:

$$(C, T) = \text{Enc}(K, N, P, A) \quad P = \text{Dec}(K, N, C, T, A) \text{ atau } \perp$$

dengan K kunci, N nonce, P plainteks, A associated data, C cipherteks, dan T tag. Simbol $\perp$ menyatakan penolakan. Sifat inilah yang membedakan AEAD dari mode enkripsi biasa: dekripsi tidak akan pernah mengembalikan plainteks bila tag tidak cocok. Pada aplikasi ini seluruh header kontainer dijadikan associated data, sehingga metadata seperti salt dan nonce ikut terlindungi walaupun tidak dienkripsi.

2.2 AES-256-GCM

Galois/Counter Mode menggabungkan mode Counter untuk enkripsi dengan fungsi hash GHASH di atas medan berhingga $GF(2^{128})$ untuk autentikasi. Keystream dibangkitkan dengan mengenkripsi pecacah:

$$C_i = P_i \oplus E_K(\text{counter}_i)$$

Karena cipherteks adalah hasil XOR antara plainteks dan keystream, dan keystream hanya bergantung pada K dan N, AES-GCM bersifat seperti stream cipher. Tag dihitung sebagai GHASH atas cipherteks dan associated data. Pemakaian ulang pasangan (K, N) sangat berbahaya karena memungkinkan pemulihan kunci GHASH dan pemalsuan tag.

2.3 ChaCha20-Poly1305

ChaCha20 adalah stream cipher berbasis operasi tambah–putar–XOR (ARX) 32 bit yang menjalankan 20 ronde atas state 4×4 word. Poly1305 adalah MAC berbasis evaluasi polinomial modulo bilangan prima $2^{130} - 5$:

$$\text{tag} = ((c_1 r^q + c_2 r^{q-1} + \dots + c_q r) \bmod (2^{130} - 5) + s) \bmod 2^{128}$$

Karena tidak memerlukan tabel substitusi, ChaCha20 berjalan dalam waktu konstan dan relatif kebal terhadap serangan kanal samping berbasis cache. Kombinasi ChaCha20-Poly1305 dibakukan dalam RFC 8439 dan menjadi salah satu cipher suite wajib pada TLS 1.3.

2.4 Fungsi Turunan Kunci Berbasis Kata Sandi

Kata sandi manusia umumnya hanya memiliki entropi 20 sampai 40 bit, jauh di bawah 256 bit yang dibutuhkan kunci AES-256. Fungsi turunan kunci (KDF) menjembatani jarak ini dengan sengaja memperlambat proses dan menambahkan salt acak:

$$K = KDF(\text{password}, \text{salt}, \text{cost})$$

PBKDF2 mengulang HMAC sebanyak c kali sehingga biaya penyerang naik c kali lipat, namun murah untuk dipercepat dengan GPU. scrypt menambahkan kebutuhan memori besar sehingga serangan paralel menjadi mahal secara perangkat keras. Argon2id, pemenang Password Hashing Competition 2015 dan dibakukan pada RFC 9106, menggabungkan ketahanan terhadap serangan memori dan serangan kanal samping. Salt acak memastikan dua pengguna dengan kata sandi sama tetap memperoleh kunci berbeda dan menggagalkan rainbow table.

2.5 Metrik Pengujian

Entropi Shannon mengukur ketidakpastian rata-rata per byte dan bernilai maksimum 8 bit/byte untuk data yang benar-benar acak:

$$H(X) = - \sum p(x_i) \log_2 p(x_i)$$

Uji chi-square menguji apakah sebaran byte cipherteks dapat dibedakan dari sebaran seragam, dengan derajat kebebasan 255 dan nilai kritis 293,25 pada taraf 5 persen:

$$\chi^2 = \sum (O_i - E_i)^2 / E_i, \quad E_i = n / 256$$

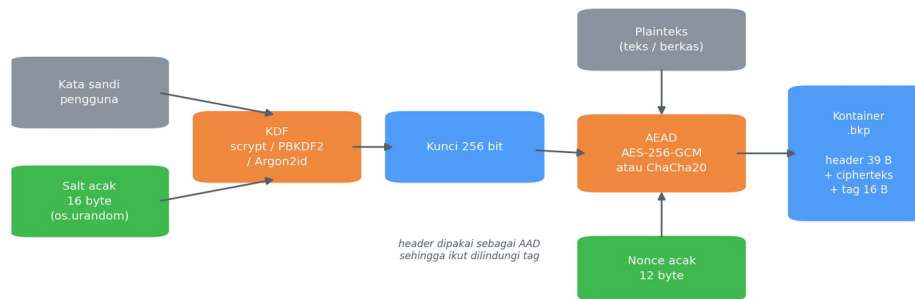
Avalanche effect mengukur persentase bit cipherteks yang berubah ketika satu bit masukan dibalik. Nilai ideal untuk difusi adalah 50 persen:

$$A = (\text{jumlah bit berbeda} / \text{total bit}) \times 100 \%$$

3. Rancangan Sistem

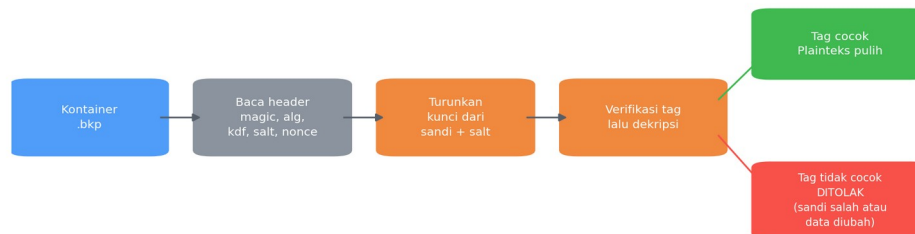
3.1 Alur Proses

Proses enkripsi dimulai dari kata sandi pengguna. Salt acak 16 byte dibangkitkan, lalu dimasukkan bersama kata sandi ke KDF untuk menghasilkan kunci 256 bit. Nonce acak 12 byte dibangkitkan terpisah, kemudian kunci, nonce, dan plainteks diumpankan ke fungsi AEAD. Keluarannya digabungkan dengan header menjadi satu container berekstensi .bcp.



Gambar 3.1 Alur proses enkripsi

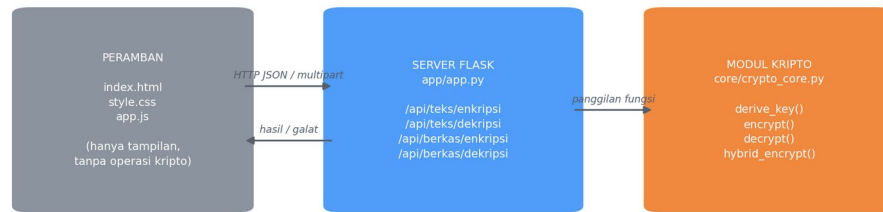
Pada proses dekripsi, header dibaca lebih dulu untuk memperoleh parameter yang dipakai saat enkripsi. Kunci diturunkan ulang dengan salt yang sama, lalu tag diverifikasi sebelum plainteks dikembalikan. Bila verifikasi gagal, aplikasi menolak tanpa mengembalikan data apa pun.



Gambar 3.2 Alur proses dekripsi dan verifikasi

3.2 Arsitektur Aplikasi

Aplikasi dibagi menjadi tiga lapisan yang terpisah tegas. Lapisan peramban hanya menangani tampilan dan tidak pernah melakukan operasi kriptografi. Lapisan server Flask menyediakan endpoint API dan menangani berkas. Seluruh operasi kriptografi terpusat pada satu modul yang tidak bergantung pada Flask, sehingga modul tersebut dapat diuji secara terpisah dan dipakai ulang di luar aplikasi web.



Kata sandi hanya hidup di memori proses server selama satu permintaan; tidak ditulis ke disk, sesi, maupun log.

Gambar 3.3 Arsitektur aplikasi

3.3 Rancangan Format Kontainer

Kontainer dirancang agar berkas hasil enkripsi dapat didekripsi tanpa menyimpan metadata apa pun di luar berkas itu sendiri. Seluruh header sepanjang 39 byte dipakai sebagai associated data.

Tabel 3.1 Struktur kontainer .bkp

Offset	Ukuran (byte)	Isi	Keterangan
0	4	Magic 'BKP1'	Penanda format dan versi
4	1	Versi	Untuk kompatibilitas ke depan
5	1	ID algoritma	1 = AES-256-GCM, 2 = ChaCha20-Poly1305
6	1	ID KDF	1 = PBKDF2, 2 = scrypt, 3 = Argon2id
7	4	Parameter biaya	Iterasi PBKDF2 atau parameter N scrypt
11	16	Salt	Acak, dari os.urandom
27	12	Nonce	Acak, unik per enkripsi
39	n	Cipherteks	Sepanjang plainteks
39 + n	16	Authentication tag	Menentukan diterima atau ditolak

3.4 Rancangan Antarmuka

Antarmuka terdiri atas dua tab. Tab Teks menampilkan plainteks dan cipherteks berdampingan dengan pilihan format Base64 atau heksadesimal serta tombol salin. Tab Berkas menyediakan unggah berkas untuk dienkripsi maupun didekripsi, ditambah tombol untuk membaca metadata kontainer tanpa kata sandi. Panel informasi di bagian bawah menampilkan algoritma, KDF, salt, dan nonce yang dipakai, sehingga penguji dapat memverifikasi bahwa nilai-nilai tersebut berubah pada setiap operasi.

« Sisipkan di sini tangkapan layar aplikasi: (a) hasil enkripsi teks, (b) pesan penolakan saat kata sandi salah, (c) tab berkas. »

4. Implementasi

Aplikasi ditulis dengan Python 3.11 dan Flask. Primitif kriptografi memakai pustaka cryptography yang telah teruji, sedangkan format kontainer, alur penurunan kunci, dan seluruh logika penolakan ditulis sendiri.

4.1 Penurunan Kunci dari Kata Sandi

```
def derive_key(password, salt, kdf_id=KDF_SCRIPT, cost=None):
    pwd = password.encode('utf-8')
    if kdf_id == KDF_SCRIPT:
        cost = cost or SCRIPT_N # N = 32768
        kdf = Script(salt=salt, length=KEY_LEN, n=cost, r=8, p=1)
        return kdf.derive(pwd), cost
```

Fungsi mengembalikan pasangan (kunci, cost) agar parameter biaya dapat disimpan di header dan dipakai ulang persis sama saat dekripsi.

4.2 Enkripsi

```
def encrypt(data, password, alg_id=ALG_AES_GCM, kdf_id=KDF_SCRIPT):
    salt = os.urandom(SALT_LEN) # 16 byte, CSPRNG sistem operasi
    nonce = os.urandom(NONCE_LEN) # 12 byte, unik setiap pemanggilan
    key, cost = derive_key(password, salt, kdf_id)

    header = Header(VERSION, alg_id, kdf_id, cost, salt, nonce)
    aad = header.pack()
    ct = _aead(alg_id, key).encrypt(nonce, data, aad)
    return aad + ct
```

Salt dan nonce dibangkitkan ulang pada setiap pemanggilan, sehingga plainteks yang sama tidak pernah menghasilkan ciphertexts yang sama. Header diteruskan sebagai associated data agar ikut dilindungi authentication tag.

4.3 Dekripsi dan Penolakan

```
def decrypt(blob, password):
    h = Header.unpack(blob)
    try:
        key, _ = derive_key(password, h.salt, h.kdf_id, h.cost)
        return _aead(h.alg_id, key).decrypt(h.nonce, blob[HEADER_LEN:],
                                             blob[:HEADER_LEN])
    except Exception as exc:
        raise CryptoError(
            'Dekripsi gagal: kata sandi salah atau data telah diubah') from exc
```

Pesan galat sengaja disamakan untuk kasus kata sandi salah dan data diubah agar tidak membocorkan informasi kepada penyerang. Seluruh alur dibungkus dalam blok try karena header yang dirusak dapat menghasilkan parameter KDF tidak valid; kasus tersebut pun harus berakhir sebagai penolakan yang terkendali, bukan galat mentah yang menghentikan aplikasi.

4.4 Enkripsi Hibrida sebagai Fitur Pengayaan

```
def hybrid_encrypt(data, public_pem):
    pub = serialization.load_pem_public_key(public_pem)
    session_key = os.urandom(KEY_LEN) # kunci sesi AES-256
    nonce = os.urandom(NONCE_LEN)
    ct = AESGCM(session_key).encrypt(nonce, data, HYB_MAGIC)
    wrapped = pub.encrypt(session_key, _OAEP) # RSA-OAEP SHA-256
    return HYB_MAGIC + struct.pack('>H', len(wrapped)) + wrapped + nonce + ct
```

Data besar tetap dienkripsi dengan kunci simetri yang cepat, sementara RSA hanya membungkus kunci sesi sepanjang 32 byte. Pola yang sama dipakai pada TLS, PGP, dan S/MIME.

4.5 Ketentuan Keamanan yang Diterapkan

- Tidak ada kunci, kata sandi, atau kunci privat yang ditulis di kode sumber; kata sandi benchmark diambil dari variabel lingkungan atau dibangkitkan acak saat dijalankan.
- Seluruh nilai acak memakai os.urandom yang merupakan CSPRNG sistem operasi.
- Mode ECB dan algoritma usang tidak dipakai sebagai fitur keamanan utama; AES-CBC dan AES-ECB hanya muncul pada skrip pengujian sebagai pembanding.
- Kunci privat RSA disimpan dalam format PKCS#8 terenkripsi.
- Berkas .pem, .key, dan .bkp dimasukkan ke .gitignore agar tidak pernah terunggah ke repositori.

5. Pengujian dan Analisis

Seluruh pengujian dijalankan otomatis melalui skrip bench/benchmark.py atas 11 berkas uji yang mencakup teks Indonesia, teks Unicode, JSON, berkas 1 byte, citra PNG, dokumen PDF, berkas biner acak berukuran 1 KB hingga 10 MB, serta berkas berpola dengan entropi rendah. Hasil lengkap tersimpan pada hasil/hasil_pengujian.xlsx.

5.1 Uji Kebenaran Dekripsi

Setiap berkas dienkripsi dengan kedua algoritma, lalu diuji tiga hal: pemulihan plainteks, penolakan kata sandi salah, dan penolakan setelah satu byte cipherteks diubah. Dari 22 kasus uji, 22 kasus lulus seluruh pemeriksaan.

Tabel 5.1 Cuplikan hasil uji kebenaran

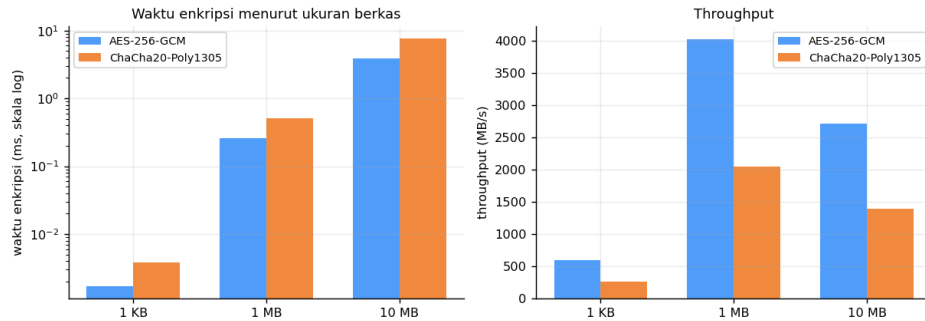
Berkas	Algoritma	Ukuran (B)	Kontainer (B)	Roundtrip	Sandi salah	1 byte diubah
02_teks_indonesia.txt	AES-256-GCM	11400	11455	BERHASIL	DITOLAK	DITOLAK
02_teks_indonesia.txt	ChaCha20-P1305	11400	11455	BERHASIL	DITOLAK	DITOLAK
06_citra.png	AES-256-GCM	4342	4397	BERHASIL	DITOLAK	DITOLAK
06_citra.png	ChaCha20-P1305	4342	4397	BERHASIL	DITOLAK	DITOLAK
07_dokumen.pdf	AES-256-GCM	866	921	BERHASIL	DITOLAK	DITOLAK
07_dokumen.pdf	ChaCha20-P1305	866	921	BERHASIL	DITOLAK	DITOLAK
10_biner_10MB.bin	AES-256-GCM	10485760	10485815	BERHASIL	DITOLAK	DITOLAK
10_biner_10MB.bin	ChaCha20-P1305	10485760	10485815	BERHASIL	DITOLAK	DITOLAK

Overhead kontainer tetap 55 byte untuk berkas berukuran apa pun, yaitu 39 byte header ditambah 16 byte tag. Pada berkas 1 byte overhead ini terasa besar secara relatif, namun menjadi tidak berarti pada berkas berukuran megabita.

5.2 Waktu Enkripsi dan Dekripsi

Tabel 5.2 Waktu enkripsi dan dekripsi (median dari 7 pengulangan)

Algoritma	Ukuran	Enkripsi (ms)	Dekripsi (ms)	Simpangan baku (ms)	Throughput (MB/s)
AES-256-GCM	1 KB	0.002	0.001	0.005	597.8
AES-256-GCM	1 MB	0.261	0.247	0.178	4024.9
AES-256-GCM	10 MB	3.854	4.564	1.555	2720.8
ChaCha20-Poly1305	1 KB	0.004	0.004	0.465	265.8
ChaCha20-Poly1305	1 MB	0.511	0.543	0.155	2054
ChaCha20-Poly1305	10 MB	7.545	6.957	1.211	1389.7



Gambar 5.1 Waktu enkripsi dan throughput

AES-256-GCM mencapai throughput 2720.8 MB/s pada berkas 10 MB, sedangkan ChaCha20-Poly1305 mencapai 1389.7 MB/s. Keunggulan AES-GCM pada mesin uji berasal dari instruksi AES-NI yang menjalankan ronde AES di perangkat keras. Pada perangkat tanpa AES-NI, misalnya telepon pintar kelas bawah atau sistem tertanam, urutan ini umumnya terbalik. Itulah alasan TLS 1.3 tetap mempertahankan ChaCha20-Poly1305 sebagai alternatif.

Tabel 5.3 Biaya fungsi turunan kunci

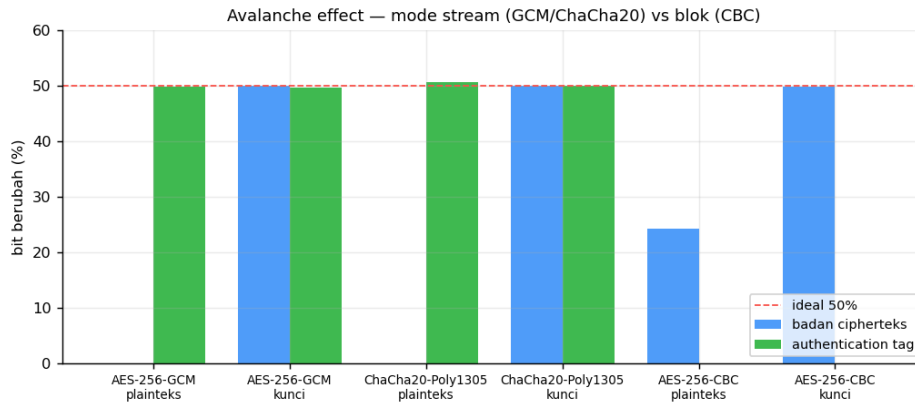
KDF	Waktu (ms)	Status
PBKDF2-HMAC-SHA256	357.41	tersedia
scrypt	133.62	tersedia
Argon2id		tidak tersedia (Argon2id butuh paket argon2-cffi)

Temuan paling penting pada pengujian waktu adalah dominasi biaya KDF. scrypt memerlukan 133.62 ms dan PBKDF2 memerlukan 357.41 ms, sedangkan enkripsi berkas 1 MB hanya 0.261 ms dengan AES-GCM. Artinya lebih dari 99 persen waktu satu operasi enkripsi berkas kecil habis di penurunan kunci. Hal ini bukan kelemahan melainkan tujuan rancangan: penyerang yang mencoba miliaran kata sandi harus membayar biaya yang sama pada setiap percobaan, sehingga serangan kamus menjadi tidak ekonomis.

5.3 Avalanche Effect

Tabel 5.4 Avalanche effect (rata-rata 100 percobaan, blok uji 1024 byte)

Algoritma / Mode	Skenario	Badan cipherteks (%)	Simpangan baku (%)	Authentication tag (%)
AES-256-GCM	1 bit plainteks diubah	0.012	0	49.875
AES-256-GCM	1 bit kunci diubah	49.996	0.523	49.727
ChaCha20-Poly1305	1 bit plainteks diubah	0.012	0	50.625
ChaCha20-Poly1305	1 bit kunci diubah	50.078	0.587	50.109
AES-256-CBC	1 bit plainteks diubah	24.19	14.354	—
AES-256-CBC	1 bit kunci diubah	49.901	0.538	—



Gambar 5.2 Avalanche effect pada mode stream dan mode blok

Hasil ini memerlukan pembacaan yang hati-hati. Ketika satu bit plainteks dibalik, badan cipherteks AES-GCM hanya berubah 0.012 persen dan ChaCha20-Poly1305 0.012 persen, yaitu praktis hanya satu bit. Angka ini jauh dari nilai ideal 50 persen, namun justru sesuai teori. Kedua mode bersifat seperti stream cipher: cipherteks merupakan hasil XOR antara plainteks dan keystream, sedangkan keystream hanya ditentukan oleh kunci dan nonce. Oleh karena itu satu bit plainteks yang berubah hanya memengaruhi satu bit cipherteks pada posisi yang sama.

Jaminan keutuhan pada skema AEAD tidak berasal dari difusi cipherteks melainkan dari authentication tag. Kolom terakhir Tabel 5.4 menunjukkan tag berubah 49.875 persen untuk AES-GCM dan 50.625 persen untuk ChaCha20-Poly1305, keduanya sangat dekat dengan nilai ideal. Inilah yang membuat seluruh percobaan manipulasi pada Tabel 5.1 berhasil ditolak.

Ketika yang dibalik adalah satu bit kunci, badan cipherteks berubah 49.996 persen pada AES-GCM dan 50.078 persen pada ChaCha20-Poly1305. Nilai yang sangat dekat dengan 50 persen ini membuktikan bahwa difusi kedua algoritma bekerja sebagaimana mestinya.

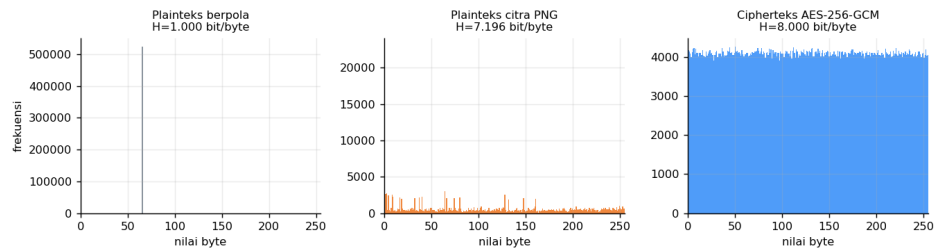
AES-256-CBC yang dipakai sebagai pembanding menghasilkan 24.19 persen pada perubahan satu bit plainteks. Angka ini mendekati 25 persen karena posisi bit yang dibalik ditentukan secara acak: rata-rata separuh blok berada sesudah titik perubahan, dan pada blok-blok tersebut sekitar separuh bit ikut berubah akibat perantaraan. Namun perlu ditegaskan bahwa difusi tinggi pada CBC sama sekali tidak memberikan jaminan keutuhan, karena CBC tanpa MAC tetap mendekripsi cipherteks yang dimanipulasi tanpa memberikan peringatan apa pun.

5.4 Entropi, Histogram, dan Uji Chi-square

Tabel 5.5 Entropi Shannon dan uji chi-square

Berkas	Algoritma	H plainteks	H cipherteks	χ^2 plainteks	χ^2 cipherteks	Kesimpulan
02_teks_indones ia.txt	AES-256-GCM	4.397	7.9833	187112.3	262	seragam
02_teks_indones ia.txt	ChaCha20-P1305	4.397	7.9835	187112.3	259.2	seragam
06_citra.png	AES-256-GCM	7.1958	7.9604	20663.8	243	seragam
06_citra.png	ChaCha20-P1305	7.1958	7.9577	20663.8	255.1	seragam
07_dokumen.pdf	AES-256-GCM	5.123	7.9594	54751	240.1	seragam
07_dokumen.pdf	ChaCha20-P1305	5.123	7.9525	54751	291	seragam
09_biner_1MB.b in	AES-256-GCM	7.9998	7.9998	223.8	244.6	seragam
09_biner_1MB.b in	ChaCha20-P1305	7.9998	7.9998	223.8	263.6	seragam

11_berpola_1M B.bin	AES-256-GCM	1	7.9998	133169152	274.4	seragam
11_berpola_1M B.bin	ChaCha20-P1305	1	7.9998	133169152	264.5	seragam



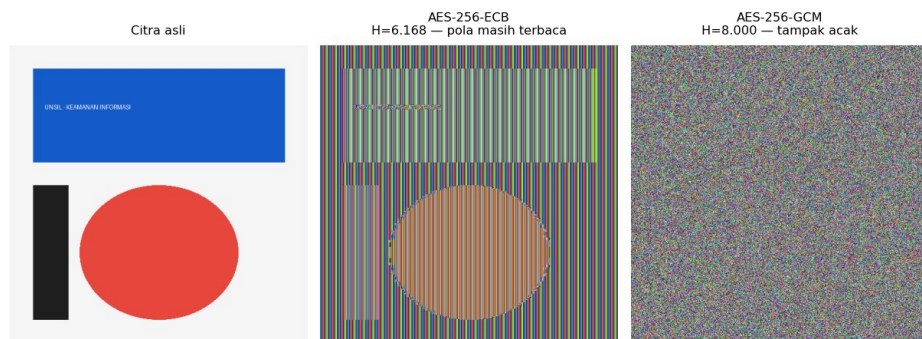
Gambar 5.3 Histogram byte plainteks dan cipherteks

Berkas berpola yang hanya berisi pengulangan delapan byte memiliki entropi 1,000 bit/byte, dan setelah dienkripsi naik menjadi 8,000 bit/byte. Teks berbahasa Indonesia naik dari sekitar 4,4 menjadi mendekati 8 bit/byte. Histogram cipherteks terlihat rata di seluruh rentang 0 sampai 255, berbeda tajam dengan histogram plainteks yang menumpuk pada beberapa nilai saja. Seluruh nilai chi-square cipherteks berada di bawah nilai kritis 293,25 pada taraf 5 persen, sehingga hipotesis bahwa sebaran byte cipherteks seragam tidak dapat ditolak. Secara statistik, cipherteks tidak dapat dibedakan dari data acak.

5.5 Perbandingan Algoritma dan Mode

Tabel 5.6 Perbandingan algoritma dan mode operasi

Algoritma / Mode	Jenis	Kunci (B)	Nonce (B)	Tag (B)	Keutuhan	Status
AES-256-GCM	AEAD (CTR + GHASH)	32	12	16	Ya	FITUR UTAMA
ChaCha20-Poly1305	AEAD (stream ARX)	32	12	16	Ya	FITUR UTAMA
AES-256-CBC	Blok, rantai	32	16	0	Tidak	PEMBANDING
AES-256-ECB	Blok, tanpa rantai	32	0	0	Tidak	PEMBANDING



Gambar 5.4 Uji visual kelemahan mode ECB pada citra

Uji visual pada Gambar 5.4 memperlihatkan alasan mode ECB dilarang. Citra yang dienkripsi dengan AES-256-ECB masih memperlihatkan bentuk persegi panjang dan elips dari citra asli, karena blok plainteks yang identik selalu menghasilkan blok cipherteks yang identik. Entropinya pun hanya sekitar

6,2 bit/byte. Sebaliknya citra yang dienkripsi dengan AES-256-GCM tampak sebagai derau acak dengan entropi mendekati 8,000 bit/byte. Contoh ini menegaskan bahwa kekuatan algoritma blok tidak ada artinya bila mode operasinya salah.

5.6 Pengujian Unit

Modul inti diuji dengan sepuluh unit test memakai pustaka unittest bawaan Python, meliputi pemulihan plainteks pada kedua algoritma, keutuhan data biner 512 KB, penolakan kata sandi salah, penolakan perubahan satu bit pada enam posisi berbeda di dalam kontainer, keunikan salt dan nonce, sifat deterministik KDF terhadap salt, pembacaan metadata, penolakan berkas asing, serta alur enkripsi hibrida RSA-OAEP. Seluruh pengujian berstatus lulus.

6. Kesimpulan dan Saran

6.1 Kesimpulan

- Aplikasi berhasil mengenkripsi dan mendekripsi teks maupun berkas dengan AES-256-GCM dan ChaCha20-Poly1305. Dari 22 kasus uji kebenaran, 22 kasus lulus seluruh pemeriksaan termasuk penolakan kata sandi salah dan penolakan manipulasi satu byte.
- Avalanche effect pada badan cipherteks kedua algoritma mendekati nol, dan hal tersebut sesuai teori karena keduanya bekerja sebagai stream cipher. Jaminan keutuhan datang dari authentication tag yang avalanche-nya terukur sekitar 50 persen.
- Entropi cipherteks mencapai 8,000 bit/byte bahkan untuk plainteks berentropi 1,000 bit/byte, dan seluruh nilai chi-square berada di bawah nilai kritis, sehingga cipherteks tidak dapat dibedakan dari data acak secara statistik.
- AES-256-GCM lebih cepat pada mesin uji karena dukungan AES-NI, namun ChaCha20-Poly1305 tetap relevan untuk perangkat tanpa akselerasi perangkat keras.
- Biaya penurunan kunci jauh melampaui biaya enkripsi itu sendiri, dan hal ini merupakan keputusan rancangan yang disengaja untuk mempersulit serangan kamus.

6.2 Saran Pengembangan

- Menerapkan enkripsi bertingkat dengan kunci utama dan kunci berkas, sehingga penggantian kata sandi tidak memerlukan enkripsi ulang seluruh berkas.
- Memproses berkas besar secara bertahap dengan skema seperti STREAM agar tidak seluruh berkas dimuat ke memori.
- Menambahkan otentikasi pengguna berbasis JSON Web Token dengan HMAC-SHA512 bila aplikasi dikembangkan menjadi layanan multipengguna.
- Mengganti kata sandi tunggal dengan skema kunci publik per pengguna, memanfaatkan modul hibrida RSA-OAEP yang sudah tersedia.
- Melakukan audit keamanan independen sebelum aplikasi dipakai untuk data sungguhan.

7. Referensi

- Bernstein, D. J. (2008). ChaCha, a variant of Salsa20. Workshop Record of SASC 2008: The State of the Art of Stream Ciphers. <https://cr.yp.to/chacha/chacha-20080128.pdf>
- Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021). Argon2 memory-hard function for password hashing and proof-of-work applications (RFC 9106). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9106>

- Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- Ferguson, N., Schneier, B., & Kohno, T. (2010). Cryptography engineering: Design principles and practical applications. Wiley.
- Katz, J., & Lindell, Y. (2020). Introduction to modern cryptography (3rd ed.). Chapman and Hall/CRC.
- Moriarty, K., Kaliski, B., & Rusch, A. (2017). PKCS #5: Password-based cryptography specification version 2.1 (RFC 8018). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8018>
- Nir, Y., & Langley, A. (2018). ChaCha20 and Poly1305 for IETF protocols (RFC 8439). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8439>
- Percival, C., & Josefsson, S. (2016). The scrypt password-based key derivation function (RFC 7914). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7914>
- Rahmatulloh, A., Gunawan, R., & Nursuwarno, F. M. S. (2019). Performance comparison of signed algorithms on JSON Web Token. IOP Conference Series: Materials Science and Engineering, 550, 012023. <https://doi.org/10.1088/1757-899X/550/1/012023>
- Rahmatulloh, A., Sulastri, H., & Nugroho, R. (2020). Web service implementation in logistics company uses JSON Web Token and RC4 cryptography algorithm. Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi), 4(3). « lengkapi volume, nomor halaman, dan DOI melalui Mendeley »
- Shannon, C. E. (1948). A mathematical theory of communication. The Bell System Technical Journal, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>

Lampiran A. Pernyataan Penggunaan Asisten AI

Sesuai ketentuan integritas akademik pada Bagian 10 petunjuk tugas, berikut rincian pemakaian asisten AI dalam pengerjaan proyek ini.

Tabel A.1 Rincian penggunaan asisten AI

Bagian Pekerjaan	Peran Asisten AI	Peran Anggota Kelompok
Rancangan format kontainer	Usulan susunan header dan pemakaian AAD	« sesuaikan: ditinjau, diuji, dan disetujui anggota »
Modul krypto inti	Kerangka kode dan komentar penjelas	« sesuaikan: verifikasi terhadap RFC dan dokumentasi pustaka »
Unit test	Usulan kasus uji termasuk kasus tepi	« sesuaikan: penambahan kasus dan penelusuran kegagalan »
Skrip pengujian	Implementasi perhitungan metrik	« sesuaikan: penentuan parameter dan jumlah percobaan »
Analisis hasil	Penjelasan teori di balik angka yang terukur	« sesuaikan: interpretasi dan penarikan kesimpulan »
Laporan	Penyusunan draf dan penataan struktur	« sesuaikan: penyuntingan, verifikasi angka, dan persetujuan akhir »

Seluruh anggota kelompok telah menelaah kode yang dikumpulkan dan mampu menjelaskan alur kerjanya, khususnya alasan pemilihan mode AEAD, fungsi salt dan nonce, serta pembacaan hasil avalanche effect pada Subbab 5.3.